

CNN, Keras & Image Classification - Key Recall Notes

Expanded from the guest-lecture deck: deep learning background, CNN intuition, transfer learning, Keras workflow, evaluation and deployment.

Big picture: image classification maps pixels -> learned features -> class probabilities. Traditional computer vision relies on hand-crafted features; deep learning learns useful representations from data.

1. Why deep learning for vision

- **Real-world images vary** in lighting, angle, scale, background, occlusion, camera type and noise.
- **Hand-written rules do not scale:** “if edge + color + texture” becomes brittle for natural images.
- **Deep learning learns features:** it discovers useful internal representations directly from examples.
- **Computer vision is spatial:** nearby pixels are related, useful patterns repeat across locations, and models must be robust to visual variation.

Recall: CNNs encode three useful ideas - locality + weight sharing + hierarchical features.

2. Neural network basics

A neural network is a sequence of mathematical transformations. Training finds weights that make predictions closer to the labels.

$$y = w x + b$$

Term	Meaning	Memory cue
Weight (w)	Learnable number controlling how strongly an input influences the output.	how much
Bias (b)	Learnable offset added after the weighted sum.	how much to shift
Activation	Non-linear function such as ReLU.	lets the network model complex patterns
Prediction	Output class probabilities, commonly from softmax in multiclass classification.	final confidence scores

3. The training loop

Step	What happens
1. Forward pass	Input flows through the model to produce predictions.
2. Loss	A loss function measures how wrong the predictions are.
3. Backpropagation	Gradients estimate how each weight contributed to the error.
4. Optimizer step	Weights are updated to reduce future loss.
5. Repeat	Train over many mini-batches while monitoring validation performance.

prediction -> loss -> gradients -> weight update -> repeat

- **Learning rate:** update step size; too high can overshoot, too low can be slow.
- **Mini-batch:** a small subset of training examples used for one update; gives a noisy but useful gradient estimate.
- **Adam:** adaptive optimizer that uses moving averages of gradients and squared gradients.

4. Model families - know the role of each

Model	Best intuition / common use
MLP	Dense connections; tabular data or simple vectors.
CNN	Images / spatial grids; local filters with shared weights.
RNN / LSTM	Sequences; hidden state carries information over time.
Transformer	Text, vision and multimodal; attention models relationships.
Autoencoder	Compression, reconstruction and anomaly detection; learns compact representations.

For this lecture: focus on CNNs and pretrained CNN backbones for image classification.

5. Keras mental model

layers -> model -> compile -> fit

Keras concept	Meaning
Layer	A transformation such as Conv2D, pooling, dropout or dense.
Model	A graph of layers with inputs and outputs.
compile()	Choose optimizer, loss and metrics.
fit()	Train the model on mini-batches.

6. Pixels to tensors

- A color image is a **3D tensor: height x width x channels**.
- Example: **224 x 224 x 3** for an RGB image.
- A batch adds another dimension: e.g. **32 x 224 x 224 x 3**.
- For 224 x 224 RGB, the model receives **150,528 numbers** per image before feature extraction.

raw pixels -> feature maps -> class probabilities

7. Kernel, filter and feature map

Term	Key meaning
Kernel	Small matrix of learnable weights (e.g. 3 x 3) that scans a local patch.
Filter	Feature detector spanning all input channels; produces one output feature map.
Feature map	Spatial map showing where the learned pattern appears strongly.
Weight sharing	The same filter is reused across the image, so the same pattern can be detected anywhere.
Locality	Each convolution looks at a small receptive field at a time.

output at one location = sum(input patch x kernel weights) + bias

Recall: number of filters = number of output feature maps / channels.

8. Hand-crafted features vs CNN features

Feature	What it captures	Memory cue
SIFT	Distinctive local keypoints and descriptors; robust to scale and rotation.	keypoints
HOG	Histogram of edge orientations; captures local shape.	edge directions
Color	Color distribution, often via RGB/HSV/Lab histograms.	color distribution
CNN filters	Features learned directly from data.	learned representations

9. Common CNN layers

Layer	What it does	Recall
Conv2D	Learns filters for edges, textures and object parts.	detect patterns
ReLU	Keeps positive activations and adds non-linearity.	non-linearity
Pooling	Shrinks feature maps while retaining strong / useful responses.	reduce spatial size
BatchNorm	Stabilizes activation statistics during training.	normalize -> scale -> shift
Dropout	Randomly disables activations during training to reduce overfitting.	regularization
Dense + Softmax	Maps learned features to class probabilities.	classifier head

10. Batch Normalization

$$x_hat = (x - \text{batch mean}) / \sqrt{\text{batch variance} + \text{epsilon}}$$

$$y = \gamma x_hat + \beta$$

- **gamma** is learnable scale; **beta** is learnable shift.
- During training it uses batch statistics; during inference it uses running estimates.
- Typical ordering: **Convolution -> BatchNorm -> ReLU**.
- Main benefit: more stable and easier optimization.

11. What CNN layers learn

Stage	Typical learned features
Early layers	Edges, colors, simple textures.
Middle layers	Corners, repeated shapes, material / texture patterns.
Late layers	Object parts and class-specific combinations.
Classifier head	Probability scores for each label.

Why transfer learning works: early and middle visual features are reusable across many image tasks.

12. Data preparation - often make-or-break

- **Folder labels:** each class can be represented by a subfolder; folder names become labels.
- **Resize:** batches need a consistent image size such as 224 x 224.
- **Split:** keep training, validation and test data separate.
- **Cache / prefetch:** speed up input loading so training is not waiting on disk.
- **Critical rule:** never evaluate on images used for training.

13. Data augmentation

- **Flip:** useful when objects may appear mirrored.
- **Rotation:** useful when camera or object may tilt.
- **Zoom:** useful when object distance varies.
- **Training only:** apply augmentation to training data, not validation/test data.
- **Domain check:** not every transform is valid; unrealistic transforms can harm learning.

Recall: augmentation increases training diversity without collecting new images.

14. Transfer learning - practical recipe

Step	Action
1. Load base	Use a pretrained backbone such as MobileNetV2, EfficientNet or ResNet.
2. Freeze base	Keep pretrained visual features unchanged initially.
3. Add head	GlobalAveragePooling + Dropout + Dense softmax.
4. Train head	Train the new classifier layers on your dataset.
5. Fine-tune	Unfreeze part / all of the base and train slowly with a much lower learning rate.

- Transfer learning is especially useful when your dataset is small.
- Use model-specific preprocessing (e.g. MobileNetV2 preprocess_input).
- Fine-tuning too aggressively can damage useful pretrained features.
- A common pattern is learning rate around **1e-3** for the frozen-head phase and a much smaller value such as **1e-5** for fine-tuning.

frozen base -> train new head -> unfreeze carefully -> fine-tune slowly

15. Overfitting and early stopping

- **Overfitting:** training performance keeps improving while validation performance gets worse.
- **Early stopping:** stop training when validation performance no longer improves.
- Possible responses: more data / better augmentation, stronger regularization, less aggressive fine-tuning, simpler model.

16. Evaluate beyond accuracy

Metric / tool	Interpretation
Accuracy	Overall fraction correct; can hide rare-class problems.
Precision	When the model predicts a class, how often is it correct?
Recall	Of all true examples of a class, how many did the model detect?
Confusion matrix	Shows which classes are confused with which.
Error inspection	Look at misclassified images and ask why.
Confidence	Top probability is not automatically trustworthy.

Class imbalance warning: a majority class can dominate accuracy, so inspect per-class metrics and the confusion matrix.

17. Debugging wrong predictions

Question	Examples
Data issue?	Wrong labels, blurry images, duplicates, class imbalance.
Preprocessing issue?	Wrong input size, channel order, or model-specific scaling.
Model issue?	Too few epochs, underfitting, overfitting, too much fine-tuning.
Task issue?	Ambiguous classes or labels that overlap conceptually.

Mindset: every wrong prediction is evidence. Use it to improve data, labels, model or deployment rules.

18. From classroom model to real-world classifier

Deployment question	Why it matters
Data shift	Will new cameras, lighting or backgrounds differ from training?
Confidence threshold	What happens when the model is unsure?
Cost of errors	Which mistakes are most harmful in the application?
Human review	Which low-confidence or high-risk predictions should go to a person?
Monitoring	How will you detect performance drift after deployment?
Ethics / privacy	Could images contain people or sensitive context?

19. Keras image-classification pipeline - one-screen recall

load folders -> resize / batch -> augment training data -> preprocess for backbone -> pretrained CNN -> pooling / dropout
-> Dense softmax -> compile -> fit -> evaluate -> inspect errors -> fine-tune if needed

20. 60-second final recall

Term	Remember this
Deep learning	Learns useful representations directly from examples.
CNN	Uses local filters, shared weights and hierarchical features.
Kernel / filter	Learnable feature detector sliding over the image.
Feature map	Where a learned pattern appears.
Weight / bias	Weight = influence; bias = shift.
BatchNorm	Normalize activations, then learn scale and shift.
SIFT / HOG / Color	Keypoints / edge directions / color distribution.
Augmentation	Increase training diversity with valid transforms.
Transfer learning	Reuse pretrained visual features; train head first.
Fine-tuning	Unfreeze carefully with a low learning rate.
Evaluation	Use confusion matrix, precision/recall and error analysis - not accuracy alone.
Deployment	Plan for uncertainty, data shift, monitoring and human fallback.

Presentation close: one improvement to the data, one to the model, and one to the evaluation.